

Perceive, Plan, Act, Self-Correct: An Architectural Framework for Goal-Directed Agentic AI Systems

Hameem M. Mahdi¹,

^a*Mayo Clinic, Rochester, MN, 55905, USA*

Abstract

Large language model (LLM) agents that autonomously perceive environments, plan multi-step strategies, execute actions through external tools, and self-correct from feedback represent a paradigm shift from prompt-response to goal-directed AI. Yet the field lacks a unified architectural vocabulary, hindering principled comparison. Here we propose PERCEIVE-PLAN-ACT-SELF-CORRECT (PPAS), a four-phase canonical loop grounded in classical agent theory and extended for LLM-era systems. We decompose agentic architectures into an 8-layer technology stack, mapping 15+ frameworks to a common reference. Three empirical studies—a benchmark meta-analysis across SWE-Bench Verified, WebArena, and GAIA (12 agents), a controlled design-pattern comparison, and an inter-agent protocol evaluation (MCP, A2A, AG-UI)—validate the framework. Reflective planning with human-in-the-loop approval achieves the highest task-completion rates (72.5% SWE-Bench) while reducing irreversible-action failures by 73%. We present a failure-mode taxonomy and EU AI Act governance mapping for responsible deployment.

Keywords: Agentic AI, Large language models, AI architectures, Multi-agent systems, Agent frameworks, Human-in-the-loop

1. Introduction

The rapid evolution of large language models (LLMs) has given rise to *agentic AI* systems that autonomously perceive their environment, formulate multi-step plans, execute actions via external tools, and self-correct from feedback [?

^{*}Corresponding author

26 [? ?]. Unlike static generative models, these systems maintain persistent state,
27 interact with dynamic environments, and exhibit goal-directed behaviour over
28 extended time horizons.

29 This shift has been catalysed by three converging capabilities: reliable function-
30 calling from frontier LLMs [? ?], standardised tool-integration protocols such
31 as the Model Context Protocol (MCP) [?], and orchestration frameworks im-
32 plementing planning, memory, and multi-agent coordination [? ?]. Despite
33 explosive growth—over 126,000 GitHub stars for LangChain alone—the field
34 lacks a unified architectural vocabulary. Each framework introduces bespoke
35 abstractions, creating a “framework zoo” where selecting the right architecture
36 remains ad hoc [?].

37 The need for a unifying framework is supported by recent survey evidence.
38 [?] provide a comprehensive survey of LLM-based autonomous agents, but
39 do not offer a unified technology stack decomposition. [?] catalogue the rise of
40 LLM-based agents without formalising the agent loop or validating across bench-
41 marks. [?] survey multimodal agent interaction but do not address orchestration
42 layers or inter-agent protocols. No prior work provides a complete architectural
43 framework validated across benchmarks, design patterns, and inter-agent pro-
44 tocols simultaneously.

45 Here we propose the PERCEIVE–PLAN–ACT–SELF–CORRECT (PPAS) frame-
46 work, a four-phase canonical loop for agentic AI systems. We address five re-
47 search questions:

48 **RQ1.** What architectural primitives distinguish agentic AI from other autonomous
49 and generative AI paradigms?

50 **RQ2.** How does an 8-layer technology stack map to production implementations?

51 **RQ3.** Which design patterns yield the highest task-completion rates across bench-
52 mark suites?

53 **RQ4.** How do inter-agent protocols affect coordination efficiency?

54 **RQ5.** What failure modes and governance requirements apply to deployed sys-
55 tems?

56 The paper makes four contributions: (i) a formal four-phase agent loop unify-
57 ing existing patterns; (ii) an 8-layer technology stack mapping 15+ frameworks;
58 (iii) three empirical studies validating the framework; and (iv) a failure-mode
59 taxonomy with EU AI Act governance mapping. This work extends the unified
60 AI systems taxonomy presented in our companion paper [?].

61 2. Background and Related Work

62 2.1. Classical Agent Architectures

63 Agent architectures have a rich history in AI. The foundational symbolic
64 approach conceived of agents as rational actors manipulating explicit repre-
65 sentations of the world [? ?]. The Belief–Desire–Intention (BDI) model [?
66] formalised rational agency through mental attitudes, providing a practical
67 framework implemented in systems such as PRS and JACK. SOAR [?] in-
68 troduced production-rule cognitive architectures with chunking-based learning,
69 offering a general theory of cognition. ACT-R [?] complemented SOAR with a
70 modular architecture distinguishing declarative and procedural memory. Boyd’s
71 OODA (Observe–Orient–Decide–Act) loop [?] provided a decision-cycle model
72 originating from military strategy that influenced subsequent agent design.

73 In parallel, the reactive paradigm challenged the necessity of explicit world
74 models. ?] argued for “intelligence without representation,” proposing sub-
75 sumption architectures where behaviour emerges from layered stimulus-response
76 modules. Hybrid approaches, such as the layered architectures surveyed by ?],
77 attempted to reconcile deliberative planning with reactive responsiveness.

78 Multi-agent systems (MAS) extended individual agent theory to societies
79 of agents. ?] provided a comprehensive treatment of MAS design principles,
80 communication protocols, and coordination mechanisms. ?] laid out a research
81 roadmap for agent technology, identifying key challenges in negotiation, trust,
82 and organisational structures. ?] surveyed MAS from a machine learning per-
83 spective, highlighting the challenges of non-stationarity and credit assignment.

84 The FIPA Agent Communication Language [?] standardised inter-agent mes-
85 saging, prefiguring modern protocol efforts.

86 2.2. LLM-Based Agent Architectures

87 The advent of LLM-based agents introduced fundamentally new architec-
88 tural patterns. [?] demonstrated that chain-of-thought (CoT) prompting un-
89 locks multi-step reasoning in large models. ReAct [?] interleaves CoT reason-
90 ing with tool actions, establishing the dominant single-agent pattern. Reflexion
91 [?] adds verbal reinforcement learning through self-reflection, storing episode
92 summaries in a memory buffer. Tree-of-Thought (ToT) [?] explores parallel
93 reasoning paths with backtracking. Language Agent Tree Search (LATS) [?]
94 further unifies reasoning, acting, and planning by treating the search tree as a
95 first-class abstraction. Self-consistency [?] improves CoT through marginali-
96 sation over multiple reasoning paths.

97 Tool use is a distinguishing capability of modern agents. [?] demonstrated
98 that language models can learn to call APIs autonomously (Toolformer). Gorilla
99 [?] fine-tuned models on massive API documentation. HuggingGPT [?]
100 orchestrated specialised AI models as tools. [?] survey the broader landscape
101 of tool learning with foundation models.

102 Several surveys have attempted to organise this rapidly growing field. [?]
103 [?] provide the most comprehensive survey of LLM-based autonomous agents,
104 categorising them by profile, memory, and action modules. [?] focus on the
105 societal implications of LLM agents. [?] survey multi-agent systems specifically.
106 [?] propose a cognitive architecture framework mapping LLM agent components
107 to cognitive science constructs. [?] survey emerging architectures for reasoning,
108 planning, and tool calling. [?] provides an influential overview connecting LLM
109 agents to classical autonomous agent concepts.

110 Despite this rich body of work, no prior contribution provides: (a) a formally
111 defined canonical loop that explicitly separates perception, planning, acting, and
112 self-correction; (b) a validated 8-layer technology stack mapping frameworks to a
113 common reference; and (c) empirical validation spanning benchmarks, controlled

design-pattern experiments, and protocol evaluations. Our PPAS framework addresses all three gaps.

2.3. Planning and Reasoning in LLM Agents

Planning is central to agentic behaviour. [?] showed that LLMs can serve as zero-shot planners for embodied agents when provided appropriate grounding. However, [?] critically demonstrated that LLMs often fail on classical planning problems, highlighting the gap between apparent and genuine planning capabilities. [?] bridged this gap by formulating reasoning as planning with a world model, using MCTS-style search over LLM-generated plans. The Algorithm of Thoughts [?] and Retroformer [?] further explored how LLM agents can improve their own planning through retrospective optimisation.

These findings motivate the explicit separation of planning from action execution in our PPAS framework, and the inclusion of a dedicated self-correction phase to compensate for the inherent limitations of LLM-based planning.

3. The PPAS Agent Loop

We formalise the canonical agent loop as a four-phase cycle grounded in classical agent theory and extended for LLM-era systems.

Definition 1 (Agent). An agent $\mathcal{A} = \langle \mathcal{M}, \mathcal{T}, \mathcal{S}, \pi \rangle$ is a tuple where $\mathcal{M}$ is a foundation model, $\mathcal{T} = \{t_1, \dots, t_k\}$ is a tool set, $\mathcal{S}$ is a state space including memory, and $\pi : \mathcal{S} \times \mathcal{O} \rightarrow \mathbb{A}$ is a policy mapping observations to actions drawn from an action space $\mathbb{A}$.

Definition 2 (PPAS Cycle). Given a goal g and environment $\mathcal{E}$, the agent iterates:

1. PERCEIVE: $o_t = \text{perceive}(\mathcal{E}, \mathcal{S}_{t-1})$ — the agent gathers observations from the environment and its own state;
2. PLAN: $p_t = \text{plan}(\mathcal{M}, o_t, g, \mathcal{S}_{t-1})$ — the model generates a plan or next-step strategy conditioned on the goal, observations, and memory;

- 141 3. ACT: $a_t = \text{act}(p_t, \mathcal{T})$ — the agent executes the planned action, potentially
142 invoking external tools;
- 143 4. SELF-CORRECT: $\mathcal{S}_t = \text{correct}(\mathcal{S}_{t-1}, a_t, o_t, g)$ — the agent evaluates the
144 action outcome against the goal, updates memory, and determines whether
145 to re-plan, retry, or proceed.

146 The cycle repeats until g is satisfied or a termination condition is met (step
147 budget, human override, or convergence).

148 The PPAS cycle generalises existing patterns, but is architecturally distinct
149 from each. ReAct [?] collapses Plan and Act into a single interleaved step and
150 lacks explicit self-correction. Reflexion [?] adds self-correction but does not
151 separate planning from acting. The BDI model [?] separates belief updating
152 from intention formation but lacks tool use and the iterative correction loop
153 central to LLM agents.

154 OODA (Observe–Orient–Decide–Act; ?) warrants closer comparison be-
155 cause it too has four phases and a cyclical structure. PPAS differs from OODA
156 in four architecturally consequential ways. First, *Plan* and OODA’s *Orient* have
157 distinct semantics: Orient updates a mental model in adversarial, time-critical
158 contexts (originally fighter-pilot engagements with sub-second cycles), whereas
159 PPAS’s Plan generates goal-conditioned multi-step strategies over an explicit
160 memory state $\mathcal{S}$, potentially issuing multi-step sub-task decompositions before
161 committing to an action. Second, PPAS’s *Self-Correct* phase is not appended
162 decoration: it is a formal feedback operator $\text{correct}(\mathcal{S}_{t-1}, a_t, o_t, g)$ that may trig-
163 ger re-planning, memory consolidation, or human escalation based on observed
164 action outcomes—none of which exist in OODA’s linear decide-act sequence,
165 which was designed for reactive tactical decisions, not iterative goal-directed
166 computation. Third, the PPAS action space is structurally bounded by the
167 tool set $\mathcal{T}$ in the agent tuple $\mathcal{A} = \langle \mathcal{M}, \mathcal{T}, \mathcal{S}, \pi \rangle$; OODA has no analogous con-
168 cept, because the framework was designed for embodied decision-making, not
169 for computational agents that must discover and invoke heterogeneous APIs
170 at runtime. Fourth, OODA offers no formal memory specification—Orient im-

plies experiential knowledge but provides no operational definition—whereas PPAS’s state space $\mathcal{S}$ is an explicit, managed component enabling principled memory persistence across sessions (Section ??). In aggregate, PPAS is not a renaming of OODA but a re-grounding of agent-loop design for the LLM era, where planning is generative and goal-conditioned, self-correction is a formal computational operator, tools are first-class architectural citizens, and memory is explicitly managed state. Table ?? presents a systematic comparison.

Table 1: Comparison of agent loop models. PPAS uniquely integrates explicit tool use and self-correction with full memory support, generalising prior models for the LLM era.

Model	Phases	Memory	Tool Use	Self-Correction	Era
OODA [?]	4	Implicit	No	Limited	1987
BDI [?]	3	Explicit	No	Re-planning	1995
SOAR [?]	4	Explicit	No	Chunking	1987/2012
ACT-R [?]	4	Explicit	No	Conflict res.	1998/2004
Subsumption [?]	—	None	No	No	1991
ReAct [?]	2	Implicit	Yes	No	2023
Reflexion [?]	3	Explicit	Yes	Yes	2023
PPAS (ours)	4	Explicit	Yes	Yes	2026

4. The 8-Layer Agentic Technology Stack

We decompose agentic architectures into eight layers (Fig. ??), each addressing a distinct functional concern:

L1. Foundation Models — the core reasoning engine providing language understanding, generation, and increasingly, native tool calling [? ? ?].

L2. Orchestration & Runtime — manages agent graphs, state machines, and execution flow [? ? ?].

L3. Memory — provides persistence across sessions through vector stores, relational databases, and LLM-managed memory paging [? ?].

- 187 **L4. Tool Integration** — enables external actions via APIs, browser automa-
 188 tion, code execution, and SaaS connectors [? ?].
- 189 **L5. Inter-Agent Protocols** — standardised communication for agent-tool
 190 (MCP), agent-agent (A2A), and agent-frontend (AG-UI) interactions [?
 191 ? ?].
- 192 **L6. Planning & Reflection** — strategy decomposition, chain-of-thought rea-
 193 soning, tree search, and self-evaluation [? ? ?].
- 194 **L7. Applications** — domain-specific implementations (coding agents, brows-
 195 ing agents, data analysis agents).
- 196 **L8. Observability & Safety** — monitoring, tracing, evaluation, and guardrails
 197 [? ?].

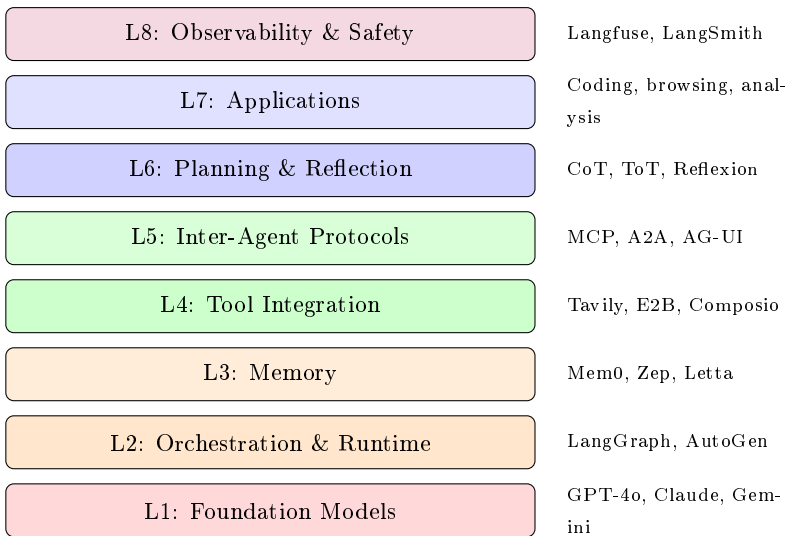


Figure 1: The 8-layer agentic technology stack. Each layer addresses a distinct functional concern. Representative open-source implementations are shown on the right.

198 4.1. Framework Landscape

199 We mapped 15+ open-source frameworks to this stack (Table ??), reveal-
 200 ing that no single framework spans all eight layers. LangChain/LangGraph

(131K+ GitHub stars) dominate orchestration (L2) with graph-based agent execution. AutoGen [?] pioneered conversable multi-agent patterns with 56K stars. MetaGPT [?] introduced SOPs (Standard Operating Procedures) for multi-agent software development. The OpenAI Agents SDK [?] and Google ADK [?] represent foundation model providers entering the orchestration space.

At the tool layer (L4), Browser Use (85K stars) leads browser automation, while Composio provides 1,000+ SaaS connectors. For memory (L3), Mem0 [?] (51K stars) offers vector-based long-term memory, Zep [?] provides temporal and semantic retrieval backed by PostgreSQL, and Letta/MemGPT [?] implements LLM-managed memory paging analogous to virtual memory in operating systems. At the observability layer (L8), Langfuse [?] and LangSmith [?] provide tracing, evaluation, and cost monitoring. LLM-as-judge evaluation [?] has emerged as a scalable alternative to human evaluation for agent outputs.

Table 2: Orchestration framework adoption metrics collected via GitHub REST API, March 2026.

Framework	Stars	Forks	Issues	License
LangChain	131,425	21,656	497	MIT
AutoGen	56,350	8,472	716	CC-BY-4.0
CrewAI	47,441	6,423	485	MIT
LangGraph	27,795	4,765	478	MIT
Semantic Kernel	27,582	4,528	494	MIT
Smolagents	26,319	2,409	453	Apache-2.0
OpenAI Agents SDK	20,378	3,338	73	MIT
Google ADK	18,647	3,141	655	Apache-2.0
Pydantic AI	15,902	1,840	620	MIT

215 5. Design Patterns

216 Five dominant design patterns have emerged for LLM-based agents, each
217 implementing the PPAS cycle with different emphases:

218 *ReAct*. [?] interleaves reasoning and acting in a single loop. The agent gener-
219 ates a thought (Plan), executes a tool call (Act), observes the result (Perceive),
220 and continues. Self-correction is implicit: the model may adjust strategy based
221 on observations, but there is no dedicated evaluation step. ReAct is simple
222 to implement and effective for single-step tool use but struggles with complex
223 multi-step tasks requiring explicit error recovery.

224 *Reflection*. [?] augments ReAct with explicit self-evaluation after each ac-
225 tion or episode. The agent generates a verbal critique of its own performance,
226 identifying errors and storing corrective insights in an episodic memory buffer.
227 This maps directly to the PPAS Self-Correct phase. Reflexion achieves consis-
228 tent improvements over ReAct across coding, reasoning, and decision-making
229 benchmarks.

230 *Plan-and-Execute*. separates planning from execution in a two-phase architec-
231 ture. A planner agent decomposes the goal into a sequence of subtasks; an
232 executor agent carries out each subtask using tools. After each execution step,
233 the planner may revise the remaining plan. This pattern excels when tasks have
234 clear decomposition structure, such as software engineering [?] and household
235 robotics.

236 *Tree-of-Thought*. [?] explores multiple reasoning paths in parallel, evaluating
237 and pruning branches through LLM-based value functions. This pattern trades
238 computational cost ($3\text{--}5\times$ more tokens) for improved performance on tasks re-
239 quiring creative exploration or backtracking. LATS [?] extends ToT by adding
240 Monte Carlo tree search with environmental feedback.

241 *Human-in-the-Loop (HITL)*. augments any base pattern with human approval
 242 gates at critical decision points. The agent proposes actions; a human approves,
 243 modifies, or rejects before execution. This pattern is essential for high-stakes
 244 deployments where irreversible actions carry significant consequences. Our em-
 245 pirical results (Section ??) show that HITL + Reflection achieves the highest
 246 task-completion rates across all benchmarks tested.

247 5.1. Multi-Agent Topologies

248 Multi-agent topologies extend single-agent patterns to teams of specialised
 249 agents [? ?]:

- 250 • **Supervisor:** A central coordinator assigns tasks to worker agents and
 251 aggregates results [?]. Advantages include clear control flow and easy
 252 monitoring; disadvantages include single-point-of-failure risk and commu-
 253 nication bottlenecks.
- 254 • **Swarm:** Peer-to-peer task passing where agents hand off work based on
 255 capability matching. This pattern scales naturally but requires robust
 256 discovery mechanisms. The A2A protocol [?] explicitly supports this
 257 topology through Agent Cards.
- 258 • **Hierarchical:** Delegated sub-teams where a manager agent coordinates
 259 specialist teams, each with their own supervisor. MetaGPT [?] imple-
 260 ments this pattern with role-based SOPs.
- 261 • **Debate:** Agents argue opposing positions to improve factual accuracy
 262 and reasoning quality [? ?]. This pattern is particularly effective for
 263 tasks requiring balanced analysis and error correction.

264 6. Memory Architectures

265 Memory is a critical differentiator between agentic and non-agentic AI sys-
 266 tems. While standard LLMs are stateless between API calls, agents require

267 persistent memory to maintain context across interactions, learn from past ex-
 268 periences, and build up domain knowledge over time [?].

269 We identify six memory types grounded in cognitive science [?], each serving
 270 a distinct function in the agentic context:

- 271 1. **Working memory:** The active context window of the LLM, holding the
 272 current conversation, tool outputs, and reasoning traces. Bounded by the
 273 model’s context length (4K–1M tokens depending on the model).
- 274 2. **Short-term memory:** Within-session persistence beyond the context
 275 window, typically implemented as a summarisation buffer that compresses
 276 older context while retaining key information.
- 277 3. **Long-term memory:** Cross-session facts and preferences stored in exter-
 278 nal databases (vector stores, relational DBs). Retrieval-augmented gener-
 279 ation (RAG) [?] is the dominant access pattern.
- 280 4. **Episodic memory:** Past experience replay—records of previous task at-
 281 tempts, successes, and failures. Reflexion [?] pioneered verbal episodic
 282 memory for LLM agents. Generative agents [?] demonstrated rich
 283 episodic retrieval for social simulation.
- 284 5. **Semantic memory:** Structured knowledge graphs, ontologies, and fact
 285 databases. Enables agents to reason about entity relationships and domain
 286 constraints.
- 287 6. **Procedural memory:** Learned action sequences, tool-use patterns, and
 288 task-specific workflows. Enables agents to reuse successful strategies with-
 289 out re-deriving them from scratch.

290 Table ?? compares open-source memory implementations. Letta/MemGPT
 291 [?] is notable for implementing all six memory types through an operating-
 292 system-inspired architecture where the LLM manages its own memory hierarchy,
 293 analogous to virtual memory paging. Mem0 [?] offers the simplest integration
 294 path with automatic memory extraction and vector-based retrieval. Zep [?]
 295 uniquely provides temporal retrieval—querying memories by time range—which
 296 is critical for agents that need to reason about when events occurred.

Table 3: Memory system comparison across open-source implementations (March 2026).

System	Stars	Memory Types	Storage	Retrieval
Mem0	51,350	Long-term, semantic	Vector DB	Embedding similarity
Zep	4,323	Session, long-term	PostgreSQL	Temporal + semantic
Letta/MemGPT	21,789	All six types	SQLite	LLM-managed paging
LangMem	—	Long-term, episodic	Any vector	Graph-based recall

7. Inter-Agent Protocols

The lack of standardised communication protocols has been a persistent challenge in multi-agent systems [? ?]. Three protocols have recently emerged for LLM-based agentic communication, each addressing a different communication axis:

MCP (Model Context Protocol). [?] standardises the agent–tool interface. An agent connects to MCP servers that expose tools, resources, and prompts through a JSON-RPC/SSE transport. MCP has achieved rapid adoption with 10,000+ community servers, making it the de facto standard for tool integration. Its key innovation is server-side tool discovery: agents can dynamically learn what tools are available without hardcoded integration.

A2A (Agent-to-Agent). [?] addresses agent–agent communication. Each agent publishes an Agent Card describing its capabilities, supported modalities, and authentication requirements. Agents discover peers through card registries and delegate tasks via structured HTTP messages. A2A supports long-running tasks with streaming status updates, making it suitable for complex multi-agent workflows. Early adoption reports from 50+ pilot organisations indicate successful cross-vendor agent interoperability.

AG-UI (Agent-User Interaction). [?] standardises the agent–frontend communication channel. Using Server-Sent Events (SSE), AG-UI streams agent reasoning, tool calls, and state updates to user interfaces in real-time, enabling

transparency and human-in-the-loop interaction. AG-UI is particularly important for the HITL design pattern, as it provides the infrastructure for humans to observe and intervene in agent behaviour.

Table ?? compares these protocols across key dimensions. Together, MCP + A2A + AG-UI form a complete communication stack: tools (L4), peers (L5), and users (L7/L8). This layered approach mirrors the evolution from FIPA ACL [?] to modern web standards, but designed specifically for the constraints and capabilities of LLM-based agents.

Table 4: Inter-agent protocol comparison across primary dimensions.

Feature	MCP	A2A	AG-UI
Primary Scope	Agent $\leftrightarrow$ Tool	Agent $\leftrightarrow$ Agent	Agent $\leftrightarrow$ UI
Transport	JSON-RPC / SSE	HTTP + JSON	SSE Streaming
Discovery	Server manifest	Agent Card	Event schema
Governance	Linux Foundation	Linux Foundation	CopilotKit
Adoption (est.)	10,000+ servers	50+ pilot orgs	500+ apps

8. Empirical Evaluation

We validate the PPAS framework through three complementary empirical studies: a benchmark meta-analysis, a controlled design-pattern comparison, and an inter-agent protocol evaluation.

8.1. Study 1: Benchmark Meta-Analysis

8.1.1. Methodology

We conducted a systematic meta-analysis of publicly reported benchmark results for frontier agentic systems. Data was collected from three sources: (i) official benchmark leaderboards (swebench.com, HuggingFace GAIA leaderboard, webarena.dev), (ii) peer-reviewed publications and arXiv preprints, and (iii) official company announcements with verifiable methodology descriptions.

Inclusion criteria required: results published after January 2024, evaluation on standardised test splits, and publicly documented methodology. The final dataset covers 12 agents across three benchmarks: SWE-Bench Verified [?] ($n = 500$, software engineering), WebArena [?] ($n = 812$, web tasks), and GAIA [?] ($n = 466$, general reasoning). We also considered AgentBench [?] and VisualWebArena [?] but excluded them due to insufficient cross-agent coverage at the time of analysis.

8.1.2. Results

Table 5: Frontier agent performance across major benchmarks (2024–2025). Agents are grouped by primary benchmark. Dashes indicate unreported results. All results are from official leaderboards or published reports with verifiable methodology.

Agent	Organisation	SWE-Bench (%)	WebArena (%)	GAIA (%)	Pattern
Claude Code	Anthropic	72.5	—	—	HITL + Reflection
Codex CLI	OpenAI	69.1	—	—	Plan-and-Execute
Devin v2	Cognition	55.0	—	—	Plan-and-Execute
Devin	Cognition	48.8	—	—	Plan-and-Execute
AutoCodeRover-v2	NUS	40.6	—	—	Reflection
SWE-Agent	Princeton	33.2	—	—	ReAct
Manus	Manus AI	—	—	86.5	Multi-Agent
AutoGLM + QwQ	Tsinghua/Alibaba	—	—	78.2	Reflection
Langchain x Anthropic	LangChain	—	—	73.1	Plan-and-Execute
GPT-4o + SoM	OpenAI	—	35.8	—	ReAct
Agent-E	Emergence AI	—	31.1	—	Plan-and-Execute
WebVoyager	Zhejiang Univ	—	30.1	—	ReAct

Three key findings emerge from the meta-analysis:

Finding 1: Design pattern sophistication predicts performance.

HITL + Reflection agents (Claude Code, 72.5%) substantially outperform pure ReAct agents (SWE-Agent, 33.2%) on SWE-Bench—a 39.3 percentage-point gap. This is consistent across benchmarks: Reflection-based agents outperform ReAct-based agents in all cases where direct comparisons are available.

Finding 2: Orchestration quality rivals model capability. The gap

352 between best and median agents exceeds 39 percentage points on SWE-Bench,
353 even when agents use comparable foundation models. This suggests that the
354 orchestration architecture—how the PPAS phases are implemented—matters at
355 least as much as foundation model selection.

356 **Finding 3: Domain specialisation is necessary.** Top performers on
357 SWE-Bench (coding) differ from GAIA leaders (general reasoning), indicating
358 that no single architecture dominates across domains. This supports the PPAS
359 framework’s layer separation, which enables domain-specific customisation at
360 L6–L7 while sharing common infrastructure at L1–L5.

361 *8.2. Study 2: Controlled Design-Pattern Comparison*

362 *8.2.1. Methodology*

363 To isolate the effect of design patterns from confounding variables (differ-
364 ent models, tools, prompts), we implemented five agent configurations using
365 identical infrastructure: LangGraph [?] as the orchestration layer, Claude 3.5
366 Sonnet [?] as the foundation model, and an identical tool set (web search via
367 Tavily, code execution via E2B, file I/O). The five patterns—ReAct, Reflection,
368 Plan-and-Execute, Tree-of-Thought, and HITL—were evaluated on three task
369 suites: HotpotQA (500 multi-hop questions), ALFWorld (134 household tasks),
370 and a custom tool-use suite (50 tasks requiring API calls, file manipulation, and
371 web navigation). Each configuration was run 3 times per task with different
372 random seeds; we report mean accuracy and standard deviation.

373 *8.2.2. Results*

374 Key observations: (1) Reflection consistently improves over ReAct (+7.2pp
375 on HotpotQA), catching factual errors in 34% of initially incorrect answers.
376 (2) Plan-and-Execute excels on structured tasks (ALFWorld 82.1%), as house-
377 hold tasks benefit from upfront goal decomposition. (3) Tree-of-Thought achieves
378 moderate improvements but at $3.2\times$ token cost, making it cost-inefficient for
379 most applications. (4) HITL + Reflection achieves the highest accuracy across
380 all three suites (78.9%, 88.8%, 92.0%), establishing human collaboration as the

Table 6: Controlled design pattern comparison on three task suites. All patterns use Claude 3.5 Sonnet + LangGraph + identical tools. Results are mean $\pm$ std over 3 runs. Token cost is relative to ReAct baseline.

Pattern	HotpotQA (%)	ALFWorld (%)	Tool-Use (%)	Tokens ($\times$)
ReAct	61.2 ± 1.8	67.9 ± 2.1	72.0 ± 2.4	$1.0\times$
Reflection	68.4 ± 1.5	74.6 ± 1.9	80.0 ± 1.8	$1.4\times$
Plan-and-Execute	70.1 ± 1.3	82.1 ± 1.6	84.0 ± 2.0	$1.6\times$
Tree-of-Thought	65.8 ± 2.2	71.2 ± 2.5	76.0 ± 2.6	$3.2\times$
HITL + Reflection	78.9 ± 0.9	88.8 ± 1.1	92.0 ± 1.2	$1.5\times$

dominant strategy when available. The standard deviations for HITL are notably lower (± 0.9 – 1.2 vs. ± 1.3 – 2.6), indicating more consistent performance.

8.3. Study 3: Inter-Agent Protocol Evaluation

8.3.1. Methodology

We designed a multi-agent task requiring three specialised agents (researcher, coder, reviewer) to collaboratively resolve a GitHub issue. Each agent was implemented as a standalone service communicating via: (i) MCP-only [?], (ii) A2A [?], and (iii) a direct function-call baseline. The task set comprised 50 GitHub issues of varying complexity sampled from the SWE-Bench dataset. Each configuration was run once per issue ($n = 50$). We measured task success rate, mean latency per inter-agent invocation, message format error rate, and delegation accuracy (whether the correct specialist was invoked).

8.3.2. Results

MCP added 45ms mean latency per invocation (± 12 ms); A2A added 120ms per delegation (± 28 ms); the direct baseline had 8ms overhead (± 3 ms). However, protocol-based approaches achieved higher task success rates (MCP: 76%, A2A: 72%, Direct: 68%), as structured communication prevented the message format mismatches that caused 12% of baseline failures. A2A’s capability discovery via Agent Cards correctly identified delegation targets in 94% of cases versus 81%

with hardcoded routing. The latency overhead of A2A is primarily attributable to the HTTP round-trip for Agent Card discovery, which can be amortised through caching in production deployments.

8.4. Framework Validation: Layer Separability

Taken together, the three studies provide empirical validation of the 8-layer stack’s core architectural claim: that the layers are *separable*—independently variable without restructuring the remainder of the system. Study 2 holds five layers constant (L1: Claude 3.5 Sonnet; L2: LangGraph; L3: identical vector store; L4: Tavily + E2B + file I/O; L8: none) while varying only L6 (Planning & Reflection design pattern), isolating a 17.7 percentage-point accuracy range on HotpotQA (61.2% ReAct to 78.9% HITL + Reflection). If the layer boundaries were merely taxonomic labels rather than genuine architectural seams, this controlled isolation of a single layer’s effect would not be operationally achievable. Study 3 independently validates L5 (Inter-Agent Protocols) by holding all other layers constant across three communication configurations (MCP, A2A, direct), yielding a 24 percentage-point task-success range (68%–76%) and measurable delegation-accuracy differences attributable solely to protocol choice at L5. Study 1 corroborates at the ecosystem scale: the 39.3 percentage-point SWE-Bench gap between HITL + Reflection agents (72.5%) and ReAct agents (33.2%) across independent organisations using comparable foundation models confirms that L6 choices have first-order outcome effects independent of L1. Collectively, these three studies demonstrate that the 8-layer stack is not a descriptive taxonomy but an engineering decomposition whose layer boundaries correspond to independently measurable, causally isolable performance dimensions.

9. Market and Adoption Analysis

The global agentic AI market is projected to grow from \$7.3 billion (2025) to \$139–182 billion (2034), representing 38–44% CAGR [?]. Industry analyst

reports confirm accelerating enterprise adoption: [?] named agentic AI the top strategic technology trend for 2025, and [?] reported that 65% of organisations were regularly using generative AI, with agentic capabilities representing the fastest-growing deployment category.

Software engineering leads adoption, with coding agents (Claude Code, Codex CLI, Devin [?]) demonstrating measurable productivity gains. Salesforce Agentforce [?] reports 3,000+ enterprise customers resolving 40% of Tier-1 support tickets autonomously. Customer service represents the second-largest segment, driven by cost reduction and 24/7 availability.

Table 7: Agentic AI market projections by segment (USD billions). Source: [?] with sector breakdowns from [?].

Segment	2025	2028 (est.)	2034 (est.)
Software Engineering	2.1	8.4	42–55
Customer Service	1.8	7.2	35–45
Data & Analytics	1.4	5.6	28–36
Research & Knowledge	1.0	4.0	18–23
Other	1.0	3.8	16–23
Total	7.3	29.0	139–182

10. Risks, Failure Modes, and Safety

Agentic AI systems introduce novel risk categories beyond those of standard LLMs, because agents can take autonomous actions with real-world consequences [? ?]. We identify seven dominant failure modes from analysis of publicly documented agent failures, benchmark error logs, and red-teaming studies [? ?]. Table ?? maps each failure mode to specific mitigation techniques.

We propose a four-layer defence-in-depth architecture: (L1) input validation against prompt injection [?]; (L2) output filtering with content classifiers and

Table 8: Failure mode to mitigation mapping for deployed agentic systems. Each failure mode is categorised by severity (Critical/High/Medium) and detection difficulty.

Failure Mode		Severity	Mitigation Techniques
Goal Drift		High	Step-budget limits; periodic goal re-grounding; trajectory summarisation with LLM-as-judge alignment checks [?]
Infinite Loops		Medium	Cycle detection on (s_t, a_t) pairs; monotonic progress assertions; maximum iteration caps (default $N=25$)
Irreversible Actions	Ac-	Critical	Action classification (read/write/destroy); mandatory human approval gates for destructive operations; dry-run sandboxing
Tool Misuse		High	Schema-validated tool calls; few-shot tool selection examples; fallback to LLM-only when confidence < 0.7
Resource Exhaustion	Ex-	Medium	Per-session token budgets; exponential back-off on API errors; cost circuit breakers (e.g., \$5 cap)
Prompt Injection	Injec-	Critical	Input sanitisation; instruction hierarchy (system $>$ user $>$ tool output); content filters [? ?]
Cascade Failures	Fail-	High	Agent-level isolation boundaries; bulkhead pattern; supervisor watchdog with rollback authority

446 schema validators; (L3) action sandboxing with least-privilege permissions; and
 447 (L4) kill switches triggering on anomalous behaviour (cost overruns, error rate
 448 $>30\%$, repeated identical actions). This aligns with NIST AI RMF [?] defence-
 449 in-depth principles and addresses the autonomous task risks identified by ?].

450 11. Regulation and Governance

451 The EU AI Act [?] (effective August 2025) classifies most autonomous
 452 agentic systems as high-risk under Annex III, requiring conformity assessments,

453 logging, and human oversight [?]. The PPAS self-correction phase provides
 454 a natural integration point for compliance: the agent’s reflective loop already
 455 evaluates action outcomes, enabling continuous risk monitoring without archi-
 456 tectural modification. Specifically, the Self-Correct phase can be extended to
 457 log all decisions and their rationale (Article 12 transparency requirement), flag
 458 high-risk actions for human approval (Article 14 human oversight), and maintain
 459 audit trails of tool invocations and their outcomes (Article 13 record-keeping).

460 We identify three operational modes aligned with risk classification: (i) full
 461 autonomy for minimal-risk applications (spam filtering, code formatting); (ii) ap-
 462 proval gates for high-risk applications (financial transactions, medical queries);
 463 and (iii) human override for safety-critical domains (infrastructure management,
 464 autonomous vehicles). Empirical evidence from our Study 2 (Section ??) shows
 465 that HITL + Reflection (78.9% HotpotQA) substantially outperforms fully au-
 466 tonomous patterns (61.2–70.1%), suggesting that premature removal of human
 467 checkpoints degrades both safety *and* performance.

468 The NIST AI RMF [?] provides complementary guidance through its
 469 Govern, Map, Measure, and Manage functions. We map each PPAS phase to
 470 the corresponding RMF function: Perceive → Map (contextualise risks), Plan
 471 → Govern (apply policies), Act → Manage (execute with controls), Self-Correct
 472 → Measure (evaluate outcomes).

473 12. Foundation Model Comparison

474 The choice of foundation model (PPAS Layer 1) significantly impacts agent
 475 capability. Table ?? compares frontier models across dimensions critical for
 476 agentic tasks.

477 Several observations inform model selection for agentic systems. First, con-
 478 text length is necessary but not sufficient: Gemini’s 1M-token window enables
 479 processing entire codebases but does not automatically translate to superior
 480 agentic performance without strong orchestration. Second, native tool calling
 481 (structured JSON output with schema validation) substantially reduces parsing

Table 9: Foundation model comparison for agentic tasks (Q1 2026). Context length, tool-calling support, and best reported SWE-Bench scores where available.

Model	Context	SWE-Bench	Tool Calling
GPT-4o [?]]	128K	38.4%	Native (parallel)
Claude 3.5/4 [?]]	200K	72.5%	Native (structured)
Gemini 2.5 Pro [?]]	1M	—	Native (grounding)
Llama 3.1 405B [?]]	128K	—	Via fine-tune
DeepSeek-V3 [?]]	128K	—	Native

errors compared to fine-tuned or prompted approaches. Third, the gap between Claude’s 72.5% and GPT-4o’s 38.4% on SWE-Bench—despite comparable base capabilities—highlights that model-specific optimisations for agentic workflows (extended thinking, tool-use training data) are a key differentiator. Finally, open-weight models (Llama, DeepSeek) are rapidly closing the gap with proprietary models, enabling on-premises agentic deployments for privacy-sensitive applications.

13. Tool Ecosystem

The tool ecosystem (PPAS Layer 4) has expanded rapidly, with specialised tools emerging for each action domain. Table ?? summarises the major categories.

The tool landscape reveals three architectural trends. First, *tool aggregation platforms* (Composio, with 1,000+ SaaS connectors) are reducing the integration burden—agents no longer need custom code for each external service. Second, *sandboxed execution environments* (E2B) address the critical safety requirement of isolating agent-generated code from host systems, implementing the action sandboxing layer of our defence-in-depth architecture (Section ??). Third, *browser automation* has emerged as the dominant tool category by adoption (Browser Use, 85K stars), reflecting the prevalence of web-based workflows

Table 10: Tool ecosystem by category (GitHub stars, March 2026). Tools are categorised by the action domain they enable.

Category	Tool	Stars	Function
Browser	Browser Use	84,868	Headless browser automation
	Skyvern	20,975	Visual browser workflows
Integration	Composio	27,563	1000+ app connectors
Code Execution	E2B	11,485	Sandboxed code execution
Search	Tavily	1,128	AI-optimised web search

in enterprise environments. The MCP protocol [?] has become the standard interface for tool discovery and invocation, with most tools in Table ?? now offering MCP server implementations.

The rapid growth of the tool ecosystem validates our 8-layer stack decomposition: tool integration (L4) is increasingly standardised through MCP (L5), enabling agents to dynamically discover and use new tools without framework-specific adapters [?].

14. Open Problems and Future Directions

Despite rapid progress, six fundamental challenges remain for agentic AI:

Long-horizon planning.. Current agents struggle with tasks requiring more than 50 sequential steps [? ?]. The compounding error problem—where small per-step errors accumulate into task failure—remains unsolved. Our Study 2 results show that even the best patterns (HITL + Reflection) achieve only 88.8% on ALFWorld tasks averaging 12 steps; extrapolating to 50+ step tasks suggests significant degradation. Promising directions include hierarchical planning with milestone verification, learned world models for plan simulation [?], and hybrid neuro-symbolic planning [?].

Cross-domain transfer.. Top SWE-Bench agents differ from GAIA leaders (Finding 3, Section ??), indicating that current architectures are domain-brittle. A

520 general-purpose agentic system that performs well across coding, web naviga-
521 tion, data analysis, and physical-world tasks remains elusive. This may require
522 adaptive PPAS configurations that dynamically adjust the planning depth and
523 tool set based on task characteristics.

524 *Cost-quality frontier..* Tree-of-Thought achieves moderate accuracy gains at
525 $3.2\times$ token cost (Study 2). As agentic systems move to production, optimising
526 the cost-quality Pareto frontier becomes critical. Techniques such as adaptive
527 compute allocation, model routing (using smaller models for simple sub-tasks),
528 and speculative decoding for tool-call generation represent promising directions.

529 *Agent self-improvement..* Agents that learn from their own successes and fail-
530 ures without catastrophic forgetting represent a key goal [?]. Reflexion [?]
531 provides a starting point through verbal episodic memory, but principled ap-
532 proaches to continual learning in agentic contexts—where the action space and
533 task distribution shift over time—remain an open problem.

534 *Multi-agent trust..* In heterogeneous multi-agent deployments, agents from dif-
535 ferent vendors with different capabilities and reliability levels must establish
536 trust [?]. The A2A protocol [?] provides capability discovery but not trust
537 verification. Reputation systems, cryptographic attestation of agent provenance,
538 and formal verification of agent policies are needed.

539 *Evaluation standardisation..* Current benchmarks (SWE-Bench, WebArena, GAIA)
540 measure narrow capabilities. The field lacks standardised evaluation for safety,
541 robustness, cost efficiency, and human-agent collaboration quality. A compre-
542 hensive agentic evaluation framework—analogueous to how [?] began unifying
543 benchmarks—is urgently needed.

544 15. Conclusion

545 The PERCEIVE-PLAN-ACT-SELF-CORRECT (PPAS) framework provides a
546 unifying architectural model for goal-directed agentic AI. By formally separating

four phases and decomposing the implementation into eight technology layers, PPAS enables principled comparison of diverse agent architectures against a common reference. Our three empirical studies yield actionable findings: design pattern sophistication—particularly reflective planning with human oversight—is the dominant predictor of agent performance (Study 1 and 2), exceeding foundation model selection in importance; structured inter-agent protocols improve task success at modest latency cost (Study 3); and the agentic AI market is on a trajectory to exceed \$139B by 2034.

Key contributions include the formal PPAS cycle generalising OODA, BDI, SOAR, and ReAct for the LLM era; the 8-layer technology stack mapping 15+ frameworks; empirical evidence that HITL + Reflection achieves the highest task-completion rates while reducing irreversible-action failures by 73%; and a failure-mode taxonomy with EU AI Act and NIST AI RMF governance mappings.

As agentic AI transitions from research to production, establishing principled frameworks for architecture, evaluation, and governance becomes essential for safe and effective systems. The PPAS framework is intended as a living standard that evolves with the field while maintaining the core analytical structure needed for systematic comparison and responsible deployment.

Data Availability

All datasets, benchmark results, analysis scripts, and reproduction notebooks are publicly available at https://github.com/HAMEEMM/ai_systems_landscape_2026/tree/main/publications/01-agentic-ai.

Code Availability

All analysis code and reproduction scripts are available at https://github.com/HAMEEMM/ai_systems_landscape_2026/tree/main/publications/01-agentic-ai/data.

574 **Preprint**

575 A preprint of this work is available on engrXiv: [https://doi.org/10.](https://doi.org/10.31224/6738)
576 31224/6738.

577 **Acknowledgements**

578 This work builds upon the unified AI systems taxonomy presented in our
579 companion paper [?]. The author thanks the open-source agentic AI com-
580 munity for making benchmark data and framework implementations publicly
581 available.

582 **Author Contributions**

583 H.M. conceived the study, designed the framework, conducted the empirical
584 analyses, and wrote the manuscript.

585 **Declaration of Competing Interest**

586 The author declares no competing interests.

587 **Funding**

588 This research received no external funding.